PART 13

Operations & Incident Response

109 Production Incidents

110 On-Call

111 Postmortems

#109

Production Incidents

WHAT IT IS

Unplanned events that degrade or break the live service.

WHY IT MATTERS

How you respond determines user trust.

EXAMPLE

Detect (alert) → Triage and declare severity (SEV1-SEV3) → Assign roles (incident commander, comms, fixer) → Mitigate first (rollback, feature-flag off, scale up) → Communicate on a status page → Resolve → Postmortem. Root-cause analysis comes after users are unblocked.

#110

On-Call

WHAT IT IS

A rotation where engineers are reachable to respond to alerts outside normal hours.

WHY IT MATTERS

Sustained reliability needs coverage without burning people out.

EXAMPLE

```
## Runbook: HighQueueDepth
1. Check Horizon: failed vs processed jobs
2. If a single job class is failing → `php artisan
    queue:pause` that queue, inspect logs
3. Scale workers: `kubectl scale deploy/worker
    --replicas=10`
```

#111

Postmortems

WHAT IT IS

A written, blameless review after an incident: what happened, why, and how to prevent repeats.

WHY IT MATTERS

Turns failures into lasting improvements.

EXAMPLE

```
# Postmortem: Checkout failures on 2026-10-12
**Impact:** 18% of checkouts failed for 47 min (~$12k)
**Timeline (UTC):** 14:02 deploy → 14:09 alert → 14:20
    rollback started → 14:49 recovered
**Root cause:** Migration added a NOT NULL column
    without default; old pods crashed on insert.
**What went well / badly:** Alert fired in 7 min /
    rollback was manual and slow
**Action items:**
- [ ] CI check for unsafe migrations (owner: tech lead,
     due: 11/15)
- [ ] One-click rollback in pipeline
```

Found this useful?

Save it for later.

Share it with a teammate.

Full guide with all code examples:
the Backend Engineering Handbook (GitHub).